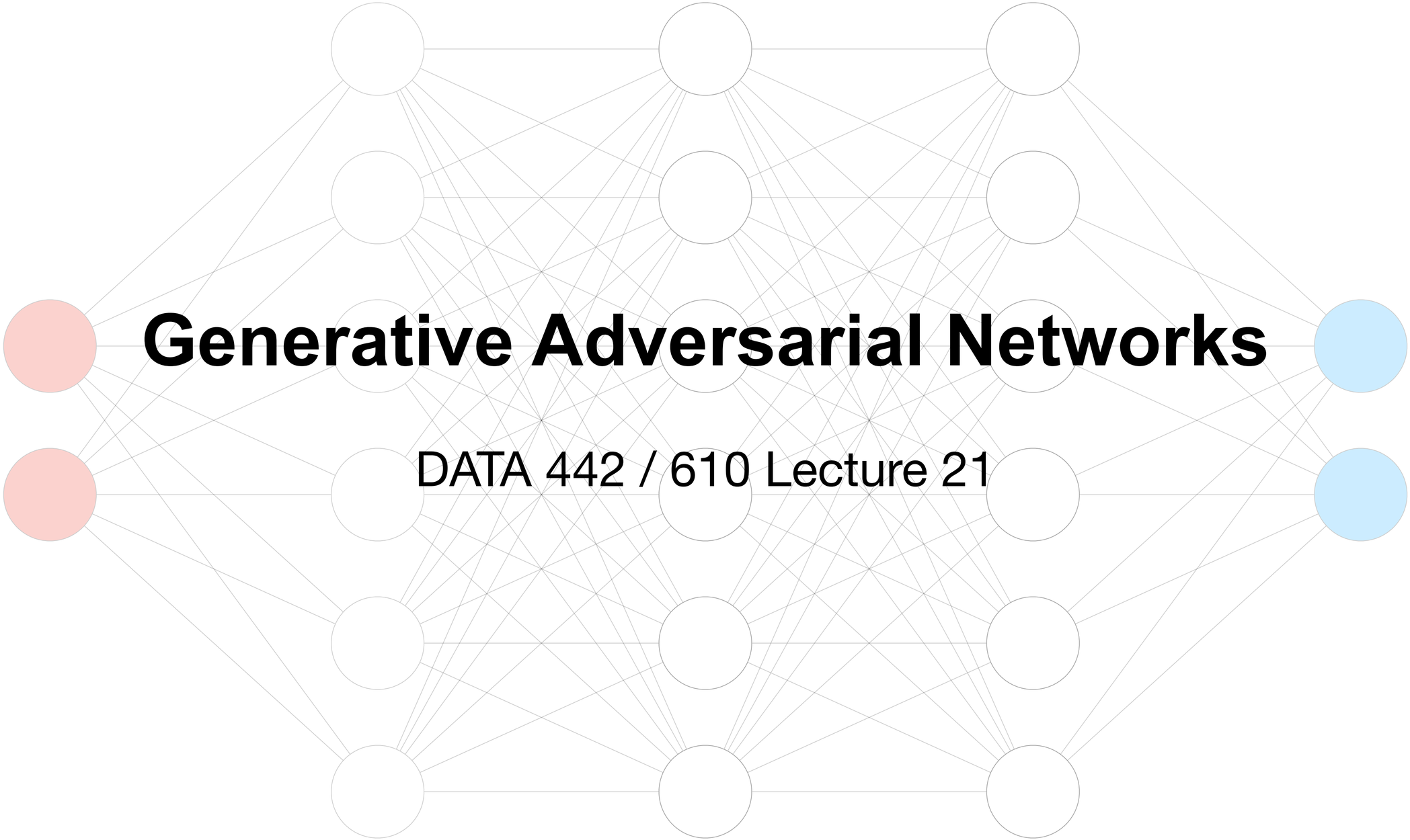


Generative Adversarial Networks

DATA 442 / 610 Lecture 21

Outline

- Review of VAEs
- Generative Adversarial Networks
- What next?

Relevant Reading: *Deep Learning* Chapter 20

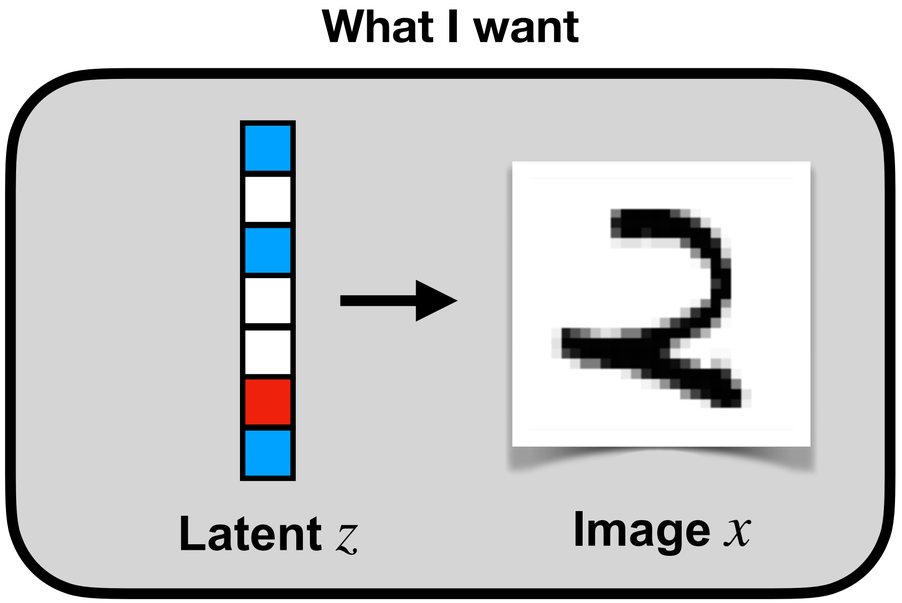
Latent generative models

Goal: data generative model $p_\theta(x)$

Strategy: try to model generation process with latent variable $z \sim p_\theta(z),\ z \to x$

$$p_\theta(x) = \int dz\ p_\theta(x, z)$$

$$= \int dz\ p_\theta(x \mid z)\, p_\theta(z)$$



Applications

1. Data generation $x \sim p_\theta(x)$
2. Latent inference $z \sim p_\theta(z \mid x)$

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z) p_\theta(z)}{p_\theta(x)}$$

Kullback-Leibler Divergence

Definition: $D_{KL}\left[P \parallel Q\right] \equiv \mathbb{E}_{x \sim p(x)}\left[\log \frac{p(x)}{q(x)}\right] \geq 0$

What is it?

A measure of distance between two distributions p, q

A measure of "surprise" when observing samples drawn from p if expecting a different distribution q

A measure of our ability to discern the "real world" p from a proposed model q

The universal recipe for learning representations

1. Draw a diagram of the data we have P
2. Augment the diagram with representations
3. Draw a diagram of how we would like to model the data and representations Q
4. Minimize the **Kullback-Leibler Divergence** $D_{KL}(P \parallel Q)$
5. Adjust seasonings, salt to taste

Variational autoencoders

$$D_{KL}\big(P \,\|\, Q\big) = D_{KL}\big(p(x)\; p_\theta(z\,|\,x)\; \|\, q(z)\, q_\phi(x\,|\,z)\big)$$

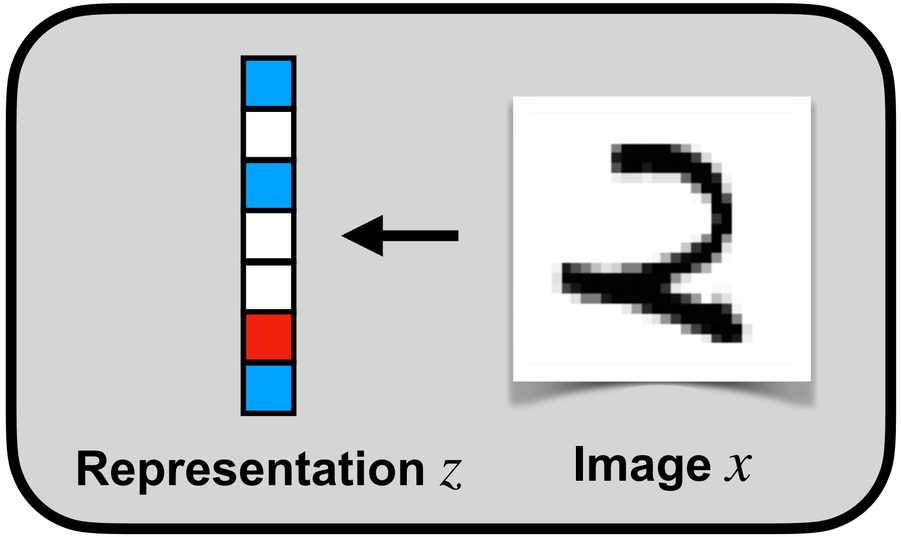
$$= \mathbb{E}_{x\sim p; z\sim p_\theta}\left[\log \frac{p(x)\, p_\theta(z\,|\,x)}{q(z)\, q_\phi(x\,|\,z)}\right]$$

$$= \cancel{\mathbb{E}_{x\sim p; z\sim p_\theta}\big[\log p(x)\big]} - \mathbb{E}_{x\sim p; z\sim p_\theta}\big[\log q_\phi(x\,|\,z)\big] + \mathbb{E}_{x\sim p; z\sim p_\theta} \log \frac{p_\theta(z\,|\,x)}{q(z)}$$

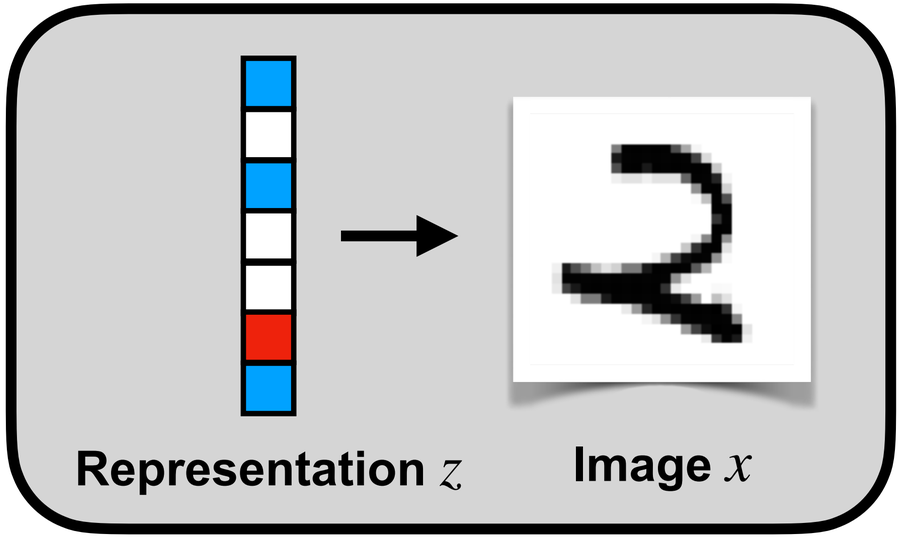
$$\mathscr{L} = -\,\mathbb{E}_{x\sim p; z\sim p_\theta}\big[\log q_\phi(x\,|\,z)\big] + D_{KL}\big[p_\theta(z\,|\,x) \,\|\, q(z)\big]$$

reconstruction **regularization**

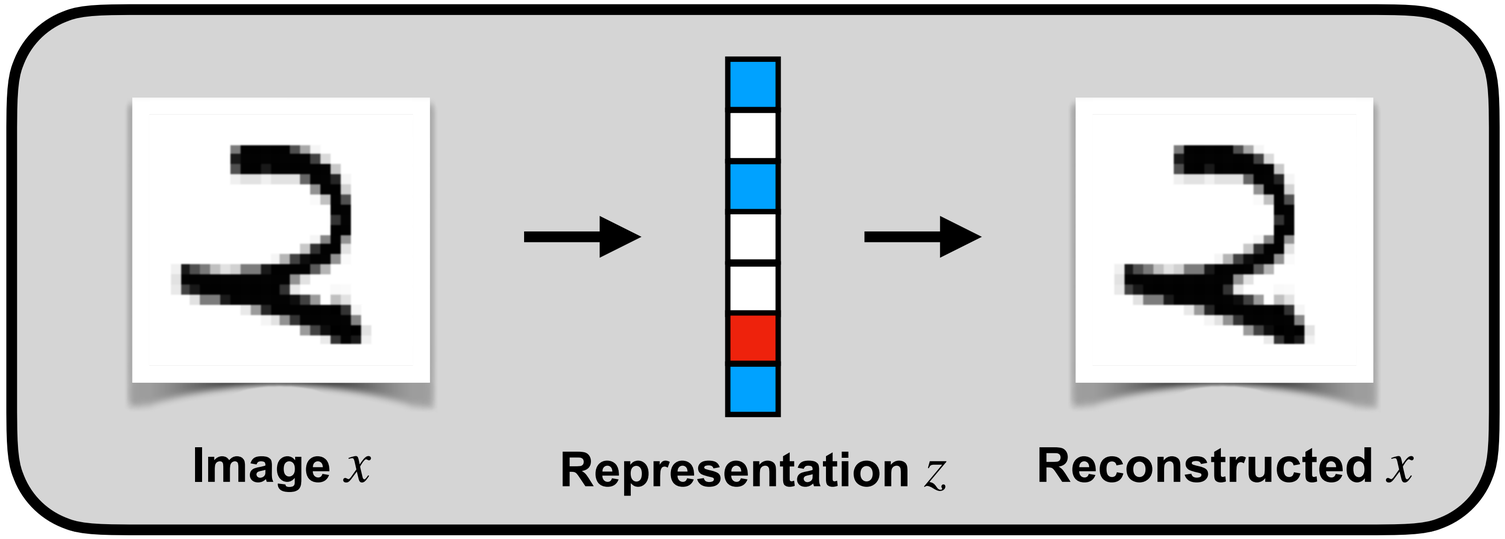
Real world P : $p(x, z) = p(x)\, p_\theta(z\,|\,x)$



Fantasy world Q : $q(x, z) = q(z)\, q_\phi(x\,|\,z)$

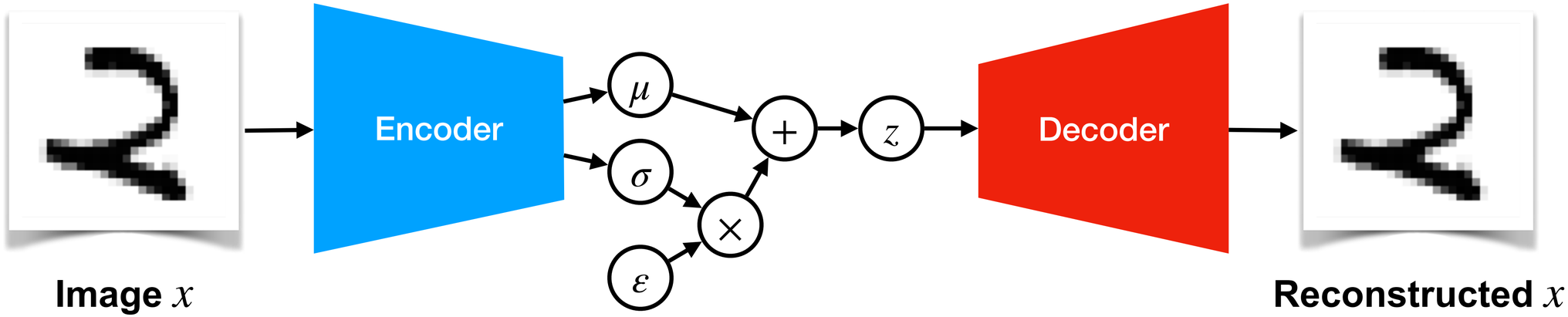


Variational autoencoders



$$\mathscr{L} = -\mathbb{E}_{x\sim p;z\sim p_\theta}\left[\log q_\phi(x\,|\,z)\right] + D_{KL}\left[p_\theta(z\,|\,x)\,\|\,q(z)\right]$$

reconstruction — regularization



Training Variational Autoencoders

Reparameterization trick

```
def forward(self, x):
    mu, logvar = self.encoder(x)

    # Sample from N(mu, sigma)
    if self.training:
        noise = torch.randn_like(mu) * (0.5 * logvar).exp()
    else:
        noise = 0.
    z = mu + noise

    x = self.decoder(z)
    return x, z, (mu, logvar)
```

"sample" $p_\theta(z \mid x)$ using auxiliary noise ε

$$z \sim p_\theta(z \mid x) = \mathcal{N}(\mu_\theta, \sigma_\theta^2)$$
$$= \mu_\theta + \sigma_\theta \varepsilon; \quad \varepsilon \sim \mathcal{N}(0,1)$$

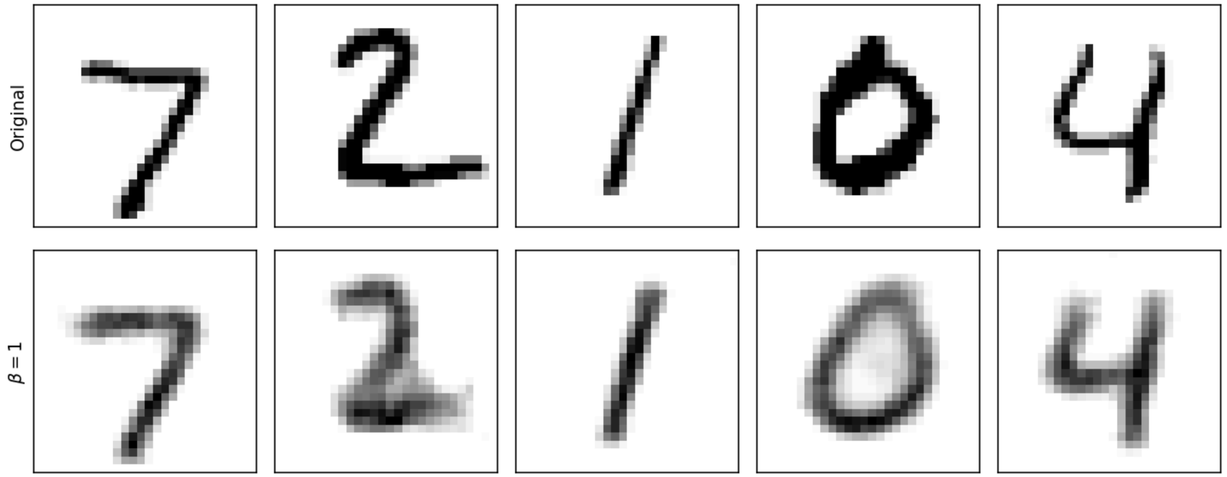
Unit Normal prior over latent variables

```
class VAELoss(nn.Module):
    def forward(self, x_pred, x_true, z_pred, mu, logvar):
        B = x_pred.shape[0]
        recon = F.mse_loss(x_pred, x_true, reduction='sum') / B
        kld = 0.5 * torch.sum( mu*mu + logvar.exp() - logvar - 1) / B
        loss = recon + kld
        return loss, {
            'loss': loss.item(),
            'recon': recon.item(),
            'kld': kld.item()
        }
```

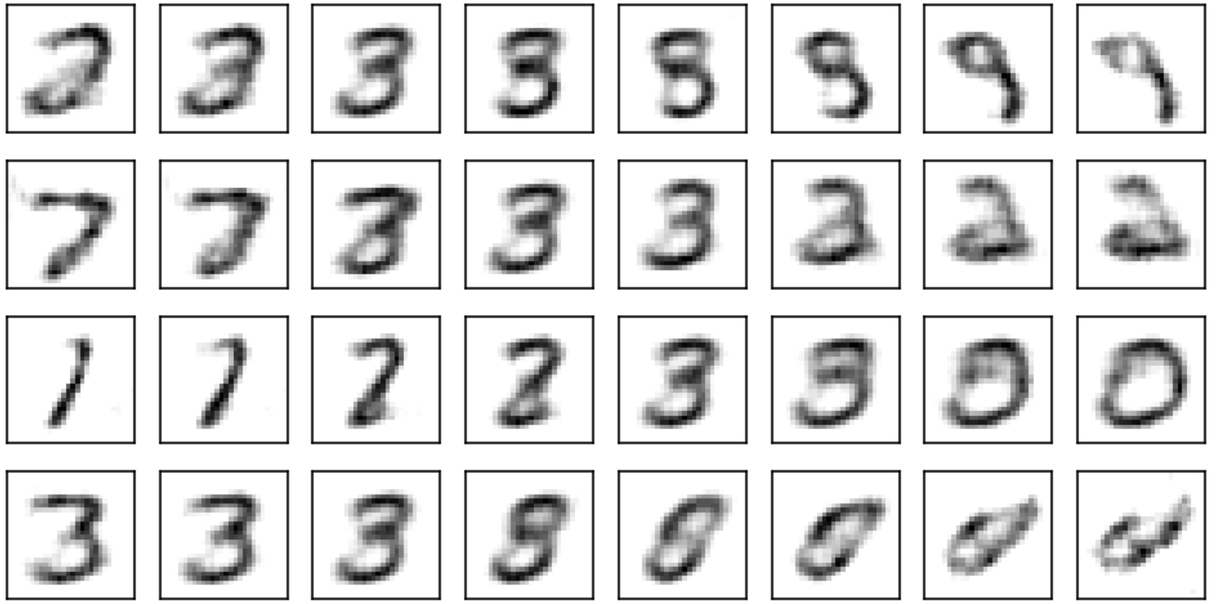
$$D_{KL}\left[\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0,1)\right] = \frac{1}{2}\left(\sigma^2 + \mu^2 - 1 - \log \sigma^2\right)$$

VAE is a latent-generative model

Generating samples from $z \sim p_\theta(z \mid x)$



Latent space traversal



Conditional VAE



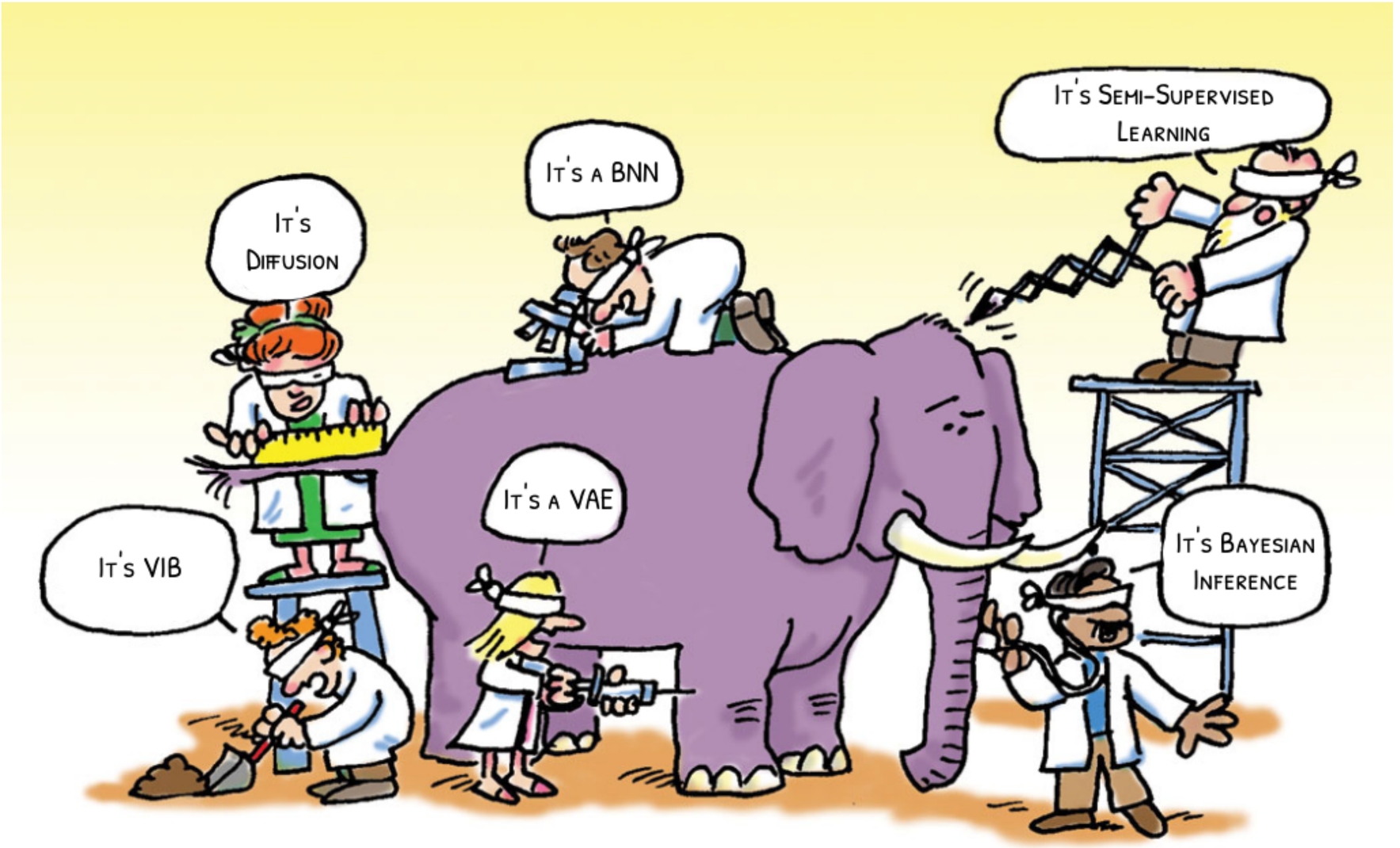


diagram shamelessly stolen from “KL is all you need,” A. Alemi

It’s $D_{KL}\left[P \,\|\, Q\right] = \mathbb{E}_P\left[\dfrac{\log P(x)}{\log Q(x)}\right] \geq 0$

The variational autoencoder is elegant, theoretically pleasing, and simple to implement…

Its main drawback is that samples from [VAEs] trained on images tend to be somewhat blurry

Deep Learning, Chapter 20, pp 694

But is KL Divergence all you need?

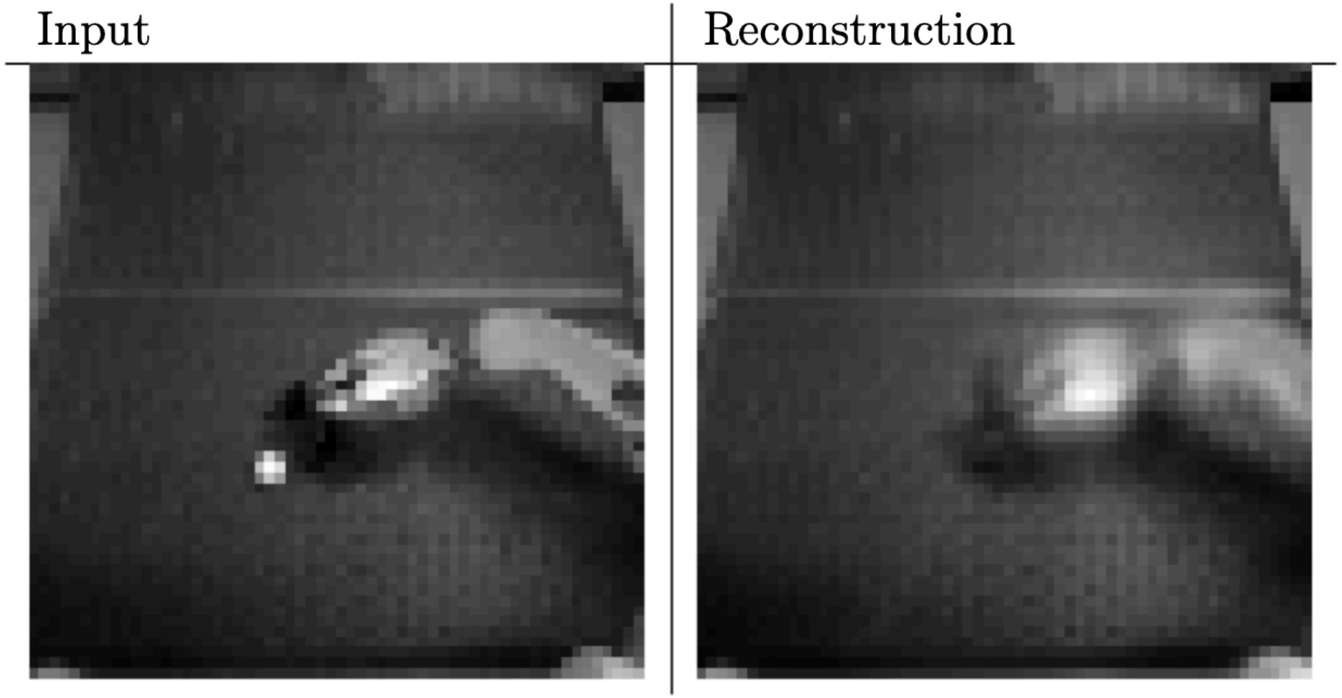


Figure 15.5: An autoencoder trained with mean squared error for a robotics task has failed to reconstruct a ping pong ball. The existence of the ping pong ball and all its spatial coordinates are important underlying causal factors that generate the image and are relevant to the robotics task. Unfortunately, the autoencoder has limited capacity, and the training with mean squared error did not identify the ping pong ball as being salient enough to encode. Images graciously provided by Chelsea Finn.

The problem

MSE comes from using a Gaussian to model the data *x*

Every pixel matters the same - important features are those that affect a lot of pixels

It's challenging to assign importance to things that only change a few pixels…

…at least, within the MLE framework

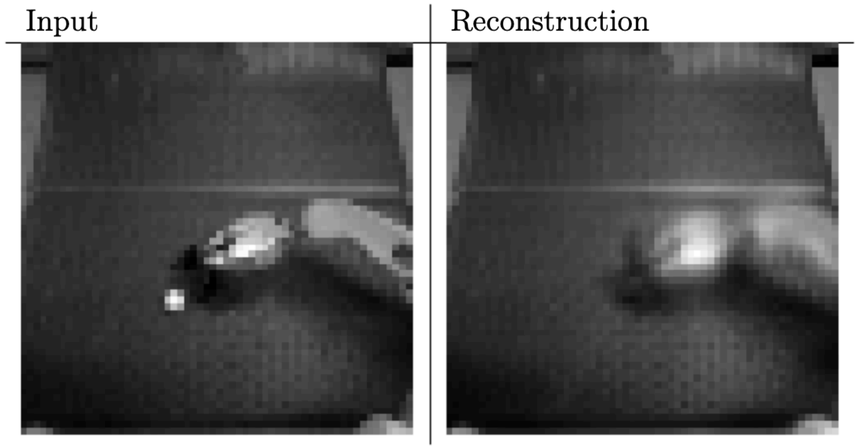


Figure 15.5: An autoencoder trained with mean squared error for a robotics task has failed to reconstruct a ping pong ball. The existence of the ping pong ball and all its spatial coordinates are important underlying causal factors that generate the image and are relevant to the robotics task. Unfortunately, the autoencoder has limited capacity, and the training with mean squared error did not identify the ping pong ball as being salient enough to encode. Images graciously provided by Chelsea Finn.

Kullback-Leibler Divergence

Minimizing the KL Divergence is an attempt to “fool” an observer into thinking our model q_θ is correct

A measure of distance between two distributions p, q

Can we make the observer a neural network?

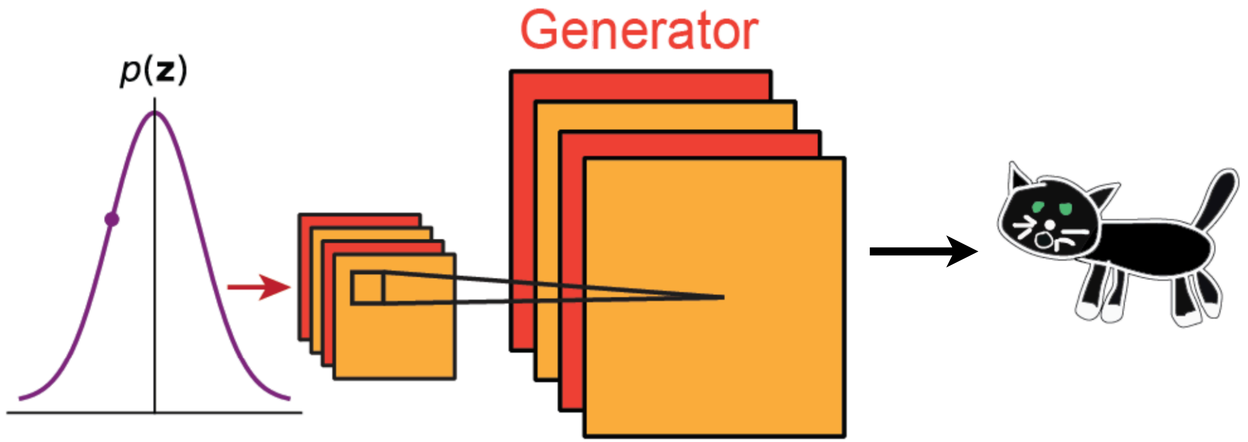
Generative Adversarial Nets

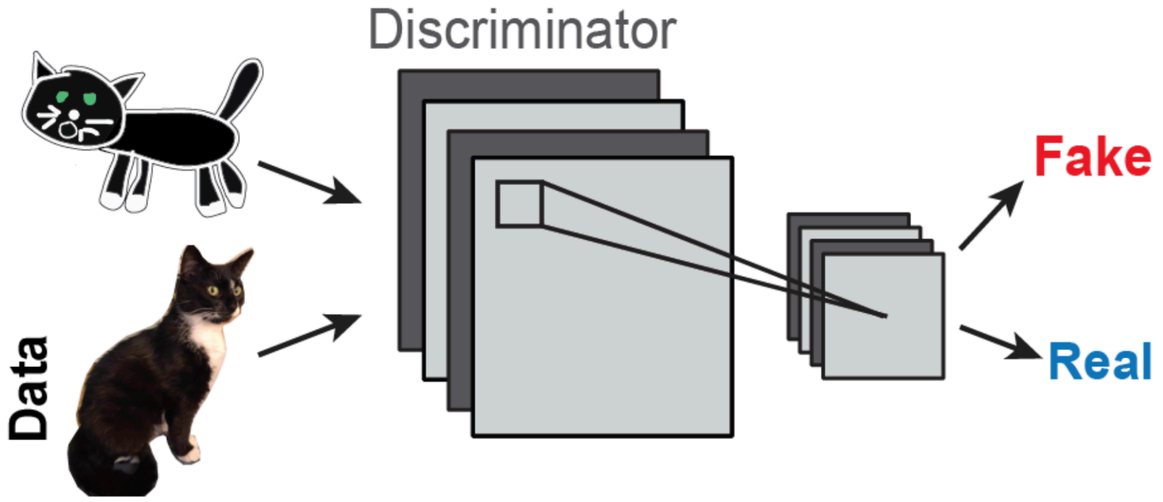
Ian J. Goodfellow, Jean Pouget-Abadie*, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair†, Aaron Courville, Yoshua Bengio‡
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G. The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D, a unique solution exists, with G recovering the training data distribution and D equal to $\frac{1}{2}$ everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.

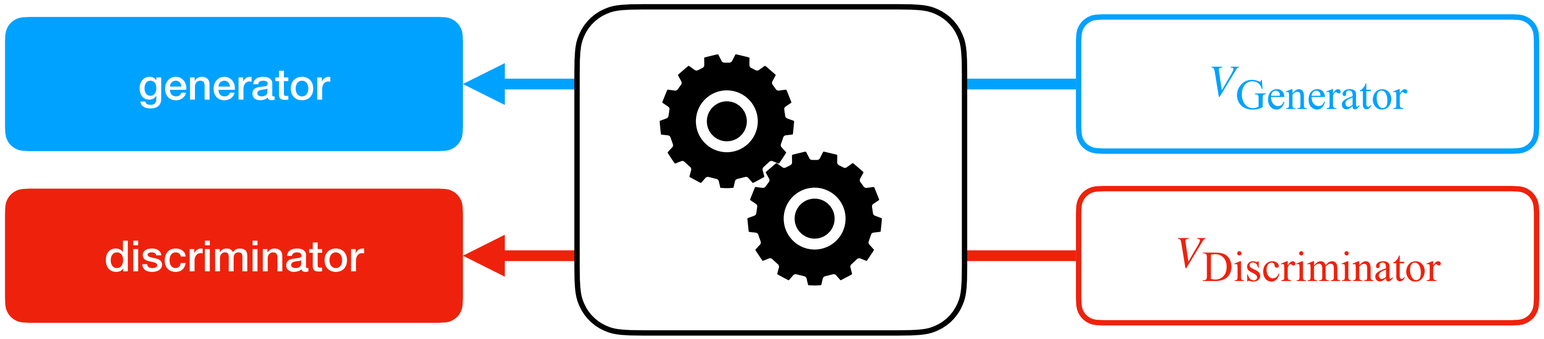
ArXiv:1406.2661



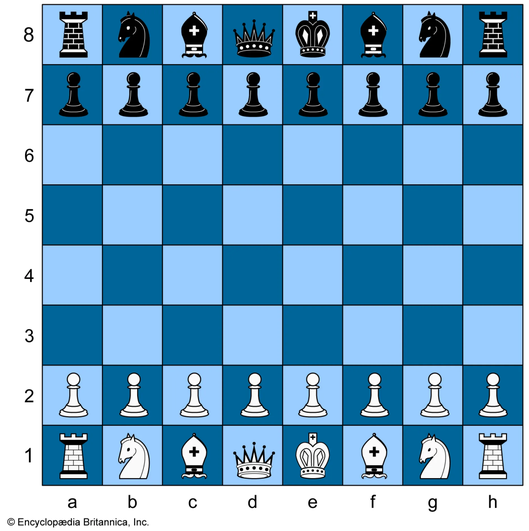


Game theory

competitors | **play the game** | **value** | **zero-sum game**



$$V_{\text{Generator}} + V_{\text{Discriminator}} = 0$$

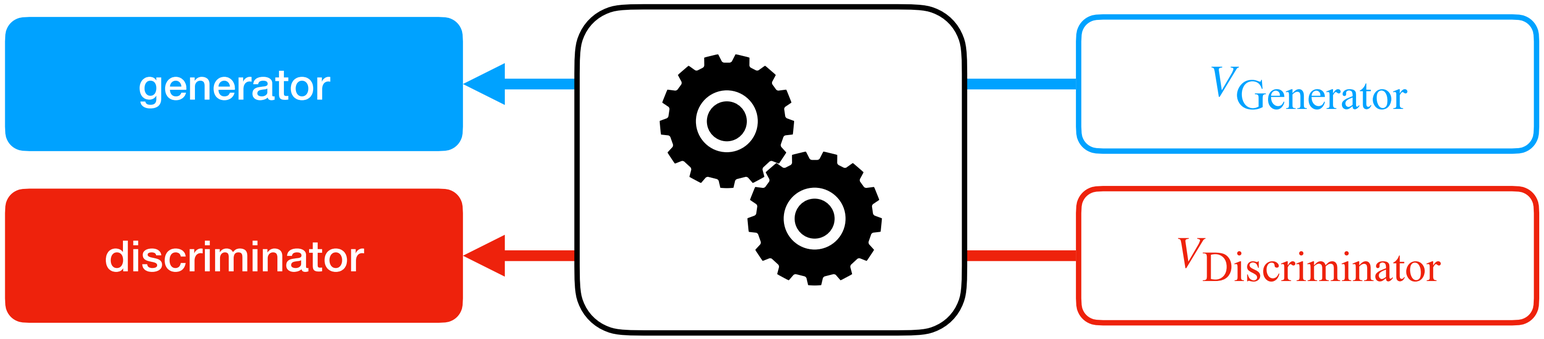




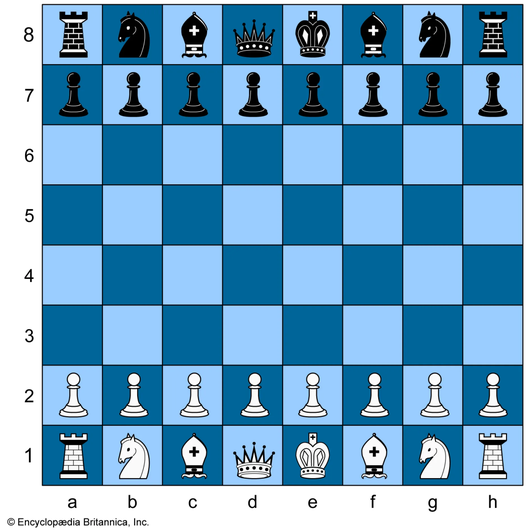


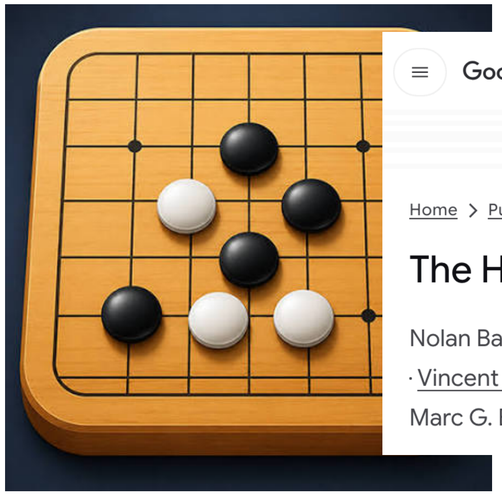
Game theory

competitors | **play the game** | **value** | **zero-sum game**



$$V_{\text{Generator}} + V_{\text{Discriminator}} = 0$$







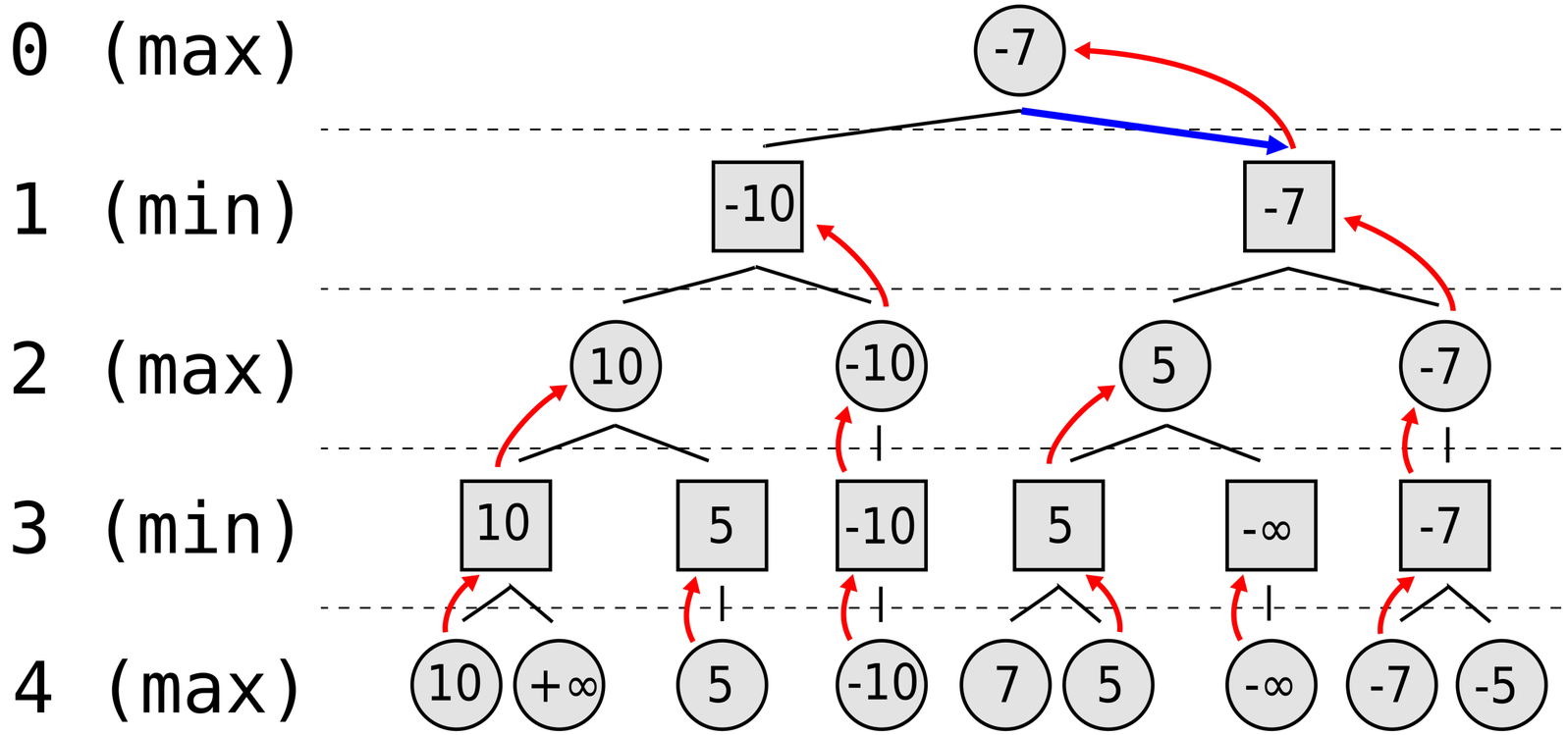
https://research.google/pubs/the-hanabi-challenge-a-new-frontier-for-ai-research/

Minimax

Minimax: $V^* = \min_G \max_D V(G, D)$

V^* is the best value D can get if it knows exactly what G will do

Turn-based game:



Generative Adversarial Networks

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p(x)}\left[\log D(x)\right] + \mathbb{E}_{z\sim p(z)}\left[\log(1 - D(G(z))\right]$$

Value function

Discriminator *recognizes* real data

Generator creates fake data

Discriminator *does not recognize* fake data

$\min_G V(D,G)$: Generator minimizes the Value function

$\max_D V(D,G)$: Discriminator maximizes the Value function

What happened to our universal recipe?

Optimal discriminator

$$D_G^*(x) = \frac{p(x)}{p(x) + p_G(x)}$$



What should the generator do?

$$\mathscr{L}_G = \max_D V(G, D) = \mathbb{E}_{x \sim p(x)}\left[\log D_G^*(x)\right] + \mathbb{E}_{z \sim p(z)}\left[\log(1 - D_G^*(G(z))\right]$$

$$= \mathbb{E}_{x \sim p(x)}\left[\log D_G^*(x)\right] + \mathbb{E}_{x \sim p_G(x)}\left[\log(1 - D_G^*(x)\right]$$

$$= \mathbb{E}_{x \sim p(x)}\left[\log \frac{p(x)}{p(x) + p_G(x)}\right] + \mathbb{E}_{x \sim p_G(x)}\left[\log \frac{p_G(x)}{p(x) + p_G(x)}\right]$$

$$= D_{KL}\left[p \,\|\, (p + p_G)/2\right] + D_{KL}\left[p_G \,\|\, (p + p_G)/2\right] - 2\log 2$$

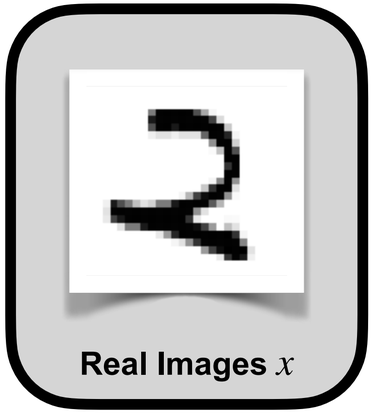
$$= 2 \cdot D_{JS}\left[p \,\|\, p_G\right] - 2\log 2$$

Jenson-Shannon divergence

What is real anymore?

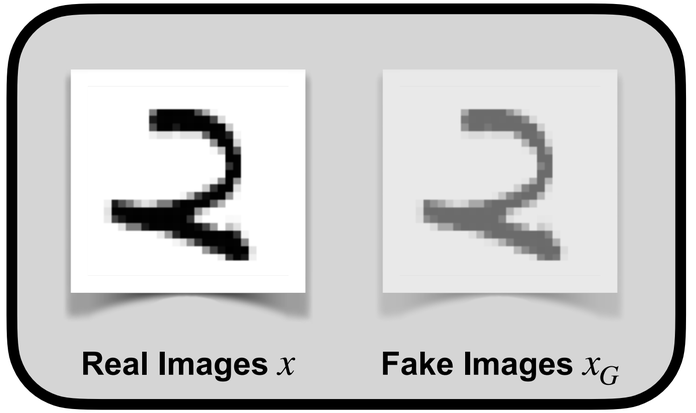
$$\mathscr{L}_G = D_{KL}\left[p \,\|\, \frac{p + p_G}{2}\right] + D_{KL}\left[p_G \,\|\, \frac{p + p_G}{2}\right] - 2\log 2$$

Fantasy world $Q : q(x) = \frac{1}{2}\left[p(x) + p_G(x)\right]$



Real world $P_1 : p(x)$

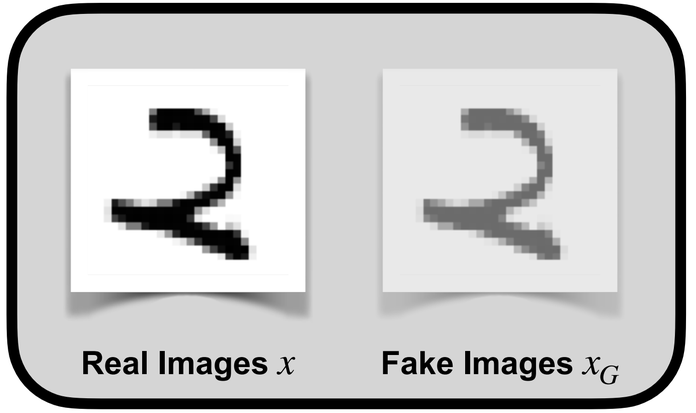






Real world $P_2 : p_G(x)$





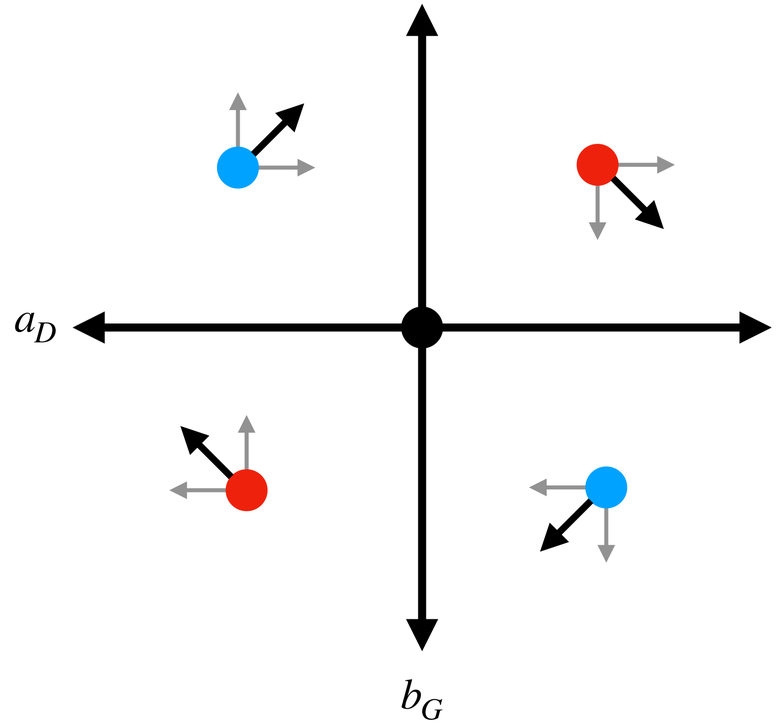
Hands-on with GANs



https://thispersondoesnotexist.com/

A challenge with GAN

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p(x)}\left[\log D(x)\right] + \mathbb{E}_{z\sim p(z)}\left[\log(1 - D(G(z))\right]$$



$V(D,G) = a_D \times b_G$

- **Global equilibrium**
- $V > 0$
- $V < 0$

GAN training is not guaranteed to converge to an equilibrium

The universal recipe for learning representations

1. Draw a diagram of the data we have P
2. Augment the diagram with representations
3. Draw a diagram of how we would like to model the data and representations Q
4. Minimize the **Kullback-Leibler Divergence** $D_{KL}(P \parallel Q)$
5. Adjust seasonings, salt to taste

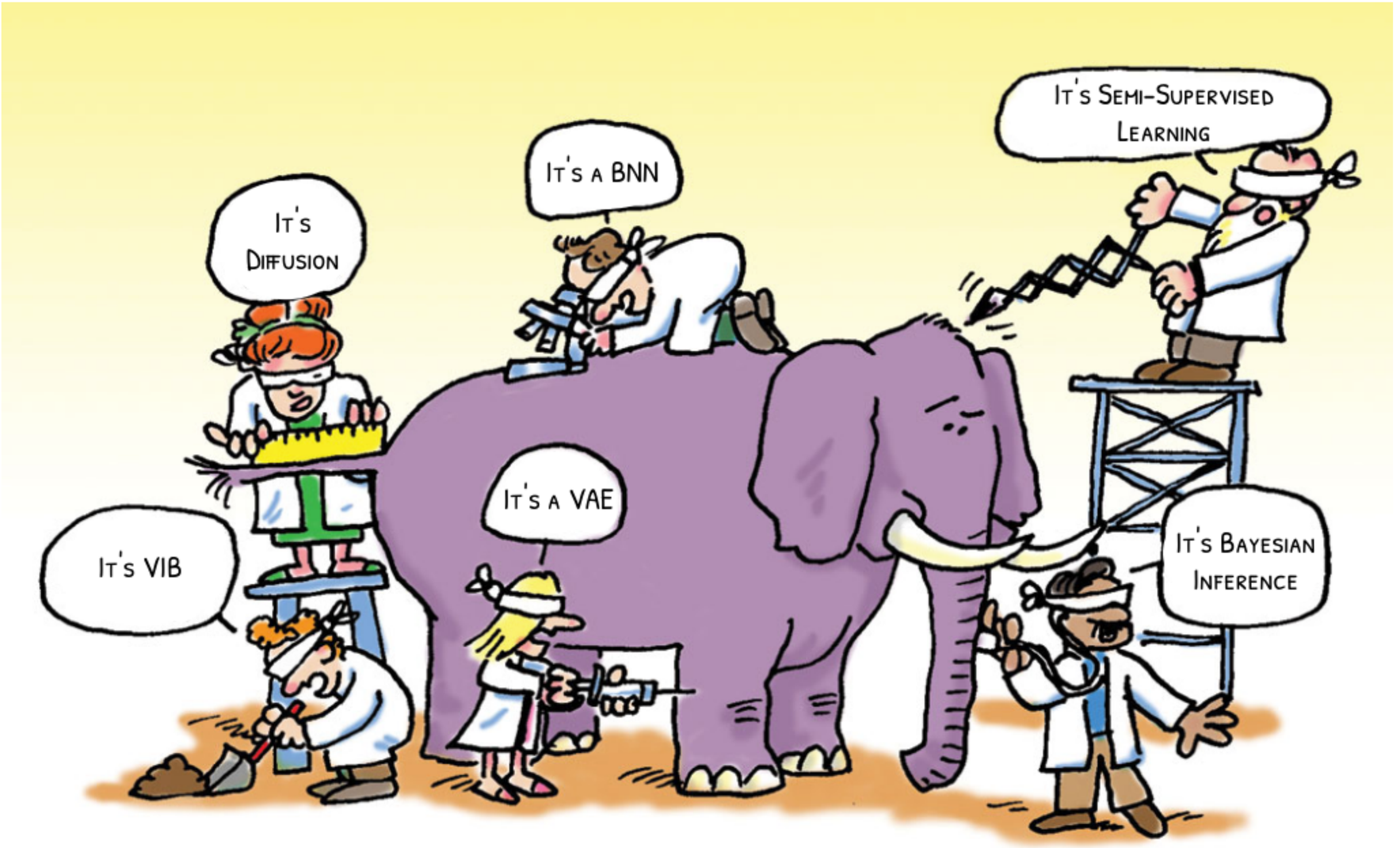


diagram shamelessly stolen from “KL is all you need,” A. Alemi

It’s $D_{KL}\left[P \,\|\, Q\right] = \mathbb{E}_P\left[\dfrac{\log P(x)}{\log Q(x)}\right] \geq 0$

That’s all!

- Slides will be posted on the course website: https://colen-teaching.github.io/DATA442-Fall-2026
- Lab demos will also be posted
- ***PS4*** due November 25
- ***Final Presentations*** on December 8, 2-5pm, ISC 2280
- **REMEMBER:** TA Office Hours Tuesday 12-1pm, Student Success Center